

Resenha Crítica: Design by Contract

Gabriel Lacerda Lemos da Silva
Engenharia de Software – 4º período

Resenha crítica

O artigo *Design by Contract*, de Bertrand Meyer, apresenta uma abordagem importante para o desenvolvimento de software confiável, principalmente no contexto da programação orientada a objetos. A ideia central do autor é comparar a construção de software com contratos do mundo real. Em um contrato, cada parte possui obrigações e benefícios bem definidos. Da mesma forma, em um sistema, uma rotina deve deixar claro o que espera de quem a chama e o que garante depois de ser executada.

Meyer parte de uma pergunta fundamental para a Engenharia de Software: como construir sistemas corretos e robustos? Essa discussão continua atual, pois muitos problemas em sistemas não acontecem apenas por falhas de tecnologia, mas por falta de clareza nas responsabilidades entre módulos, classes e métodos. O autor critica o fato de que a orientação a objetos muitas vezes valoriza reutilização, modularidade e flexibilidade, mas nem sempre dá a mesma atenção à correção e à robustez do software.

A proposta de *Design by Contract* se apoia em três conceitos principais: pré-condições, pós-condições e invariantes de classe. As pré-condições representam aquilo que deve ser verdadeiro antes da execução de uma rotina. Ou seja, são responsabilidades de quem chama o método. As pós-condições indicam aquilo que a rotina deve garantir ao final de sua execução. Já os invariantes são propriedades que devem permanecer verdadeiras para os objetos de uma classe em seus estados observáveis. Com isso, o comportamento esperado do sistema se torna mais explícito e organizado.

Um dos pontos mais interessantes do artigo é a crítica à programação defensiva em excesso. Meyer argumenta que verificar todas as condições em todos os lugares pode aumentar a complexidade do código e, conseqüentemente, diminuir sua confiabilidade. Para o autor, o ideal é definir exatamente quem é responsável por cada verificação. Se uma condição faz parte da pré-condição de uma rotina, então o cliente deve garanti-la antes da chamada. Caso contrário, a própria rotina deve tratar a situação. Essa separação evita verificações duplicadas e deixa o projeto mais limpo.

Apesar disso, é importante observar que essa ideia precisa ser aplicada com cuidado em sistemas reais. Em uma API pública, por exemplo, não é possível confiar totalmente nos dados

recebidos de usuários ou sistemas externos. Nesses casos, validações continuam sendo necessárias. Porém, a contribuição de Meyer está em mostrar que essas validações devem ter um lugar correto na arquitetura, evitando repetições desnecessárias e responsabilidades confusas.

Outro aspecto relevante é o uso das asserções como forma de documentação. Para Meyer, pré-condições, pós-condições e invariantes não servem apenas para detectar erros durante a execução, mas também para explicar o comportamento esperado de uma classe ou rotina. Isso torna a documentação mais próxima do código e facilita a manutenção. Em vez de depender apenas de comentários soltos, o próprio código passa a registrar as regras que devem ser respeitadas.

O artigo também discute a relação entre contratos e tratamento de exceções. Segundo Meyer, exceções não devem ser usadas como fluxo normal do programa, mas apenas para situações realmente anormais, quando uma das partes não consegue cumprir sua parte do contrato. Essa visão é importante porque evita transformar exceções em uma forma confusa de controle de fluxo. Em linguagens atuais, como Java, JavaScript e Python, esse problema ainda aparece quando exceções são usadas para tratar situações previsíveis que poderiam ser resolvidas com validações comuns.

No contexto da orientação a objetos, a ideia de contrato também ajuda a compreender melhor herança e ligação dinâmica. Quando uma subclasse redefine um método de uma superclasse, ela não deveria quebrar o contrato esperado pelos clientes daquela classe. Isso se relaciona com princípios importantes de projeto de software, como a substituição correta entre classes pai e filhas. Assim, o artigo mostra que contratos não são apenas detalhes de implementação, mas também ajudam a manter a coerência da arquitetura orientada a objetos.

Uma limitação do texto é que ele se baseia bastante na linguagem Eiffel, que possui suporte nativo a contratos. Como essa linguagem não é tão comum atualmente, algumas ideias precisam ser adaptadas para tecnologias modernas. Mesmo assim, os conceitos continuam úteis. Em projetos atuais, contratos podem aparecer por meio de validações de entrada, testes automatizados, interfaces, tipos, schemas e documentação de APIs.

Portanto, *Design by Contract* é um artigo relevante para a Engenharia de Software porque mostra que a confiabilidade deve ser pensada desde o projeto do sistema, e não apenas durante os testes. A principal contribuição de Meyer é mostrar que um software confiável depende de responsabilidades claras entre seus componentes. Mesmo sendo um texto antigo e ligado à linguagem Eiffel, suas ideias continuam aplicáveis em sistemas modernos, principalmente quando se busca código mais organizado, previsível e fácil de manter.

Referência

MEYER, Bertrand. *Design by Contract*. Interactive Software Engineering Inc.